

Blockchain Midterm Revision Sheet

1) First, what kind of exam are you preparing for?

From your sir's message, the exam is mostly:

- **Conceptual / short questions**
- **Simulation / numerical / step-by-step execution**
- **One objective / table / fill-in type question**

And the topics are specifically around:

- Merkle Tree and Merkle Proof
- Bitcoin transaction signature construction and verification
- Bitcoin UTXO unlocking
- Proof-of-work and mining
- UTXO database
- Transaction fee and fee rate
- Bitcoin transaction types: P2PKH, P2MS, P2SH, P2WPKH, P2WSH, P2TR

That matches the Bitcoin-focused lecture block in the outline very closely.

So do **not** study everything equally. The heavy focus should be on:

1. **How a Bitcoin transaction works**
 2. **How a UTXO is spent**
 3. **How scripts unlock outputs**
 4. **How mining / target / nonce works**
 5. **How Merkle proof works**
 6. **How transaction types differ**
-

2) Big picture: what is Bitcoin doing?

Bitcoin is not account-based like a normal bank. It does **not** mainly store "Alice balance = X, Bob balance = Y" the way Ethereum's account model does. Instead, Bitcoin uses the **UTXO model**, where transactions consume old outputs and create new outputs. The lectures explicitly contrast account-based transactions with Bitcoin's UTXO model, and also explain that replay

protection in Bitcoin comes from the UTXO model rather than Ethereum-style sequence tracking.

Super simple way to think of it

A Bitcoin output is like a locked coin.

When you spend it:

- you **unlock** an old output
- then you **create new locked outputs**

That is why the lectures say a Bitcoin can only be spent once, and whenever it is spent, new coins/outputs are generated.

3) Core foundations you must know before the Bitcoin parts

A. Cryptographic hashing

A hash function takes input of any size and produces a fixed-size, random-looking output. A tiny change in the input causes a big change in the output. That avalanche effect is one of the reasons blockchain data is tamper-evident.

Why this matters in blockchain

Each block links to the previous block using its hash. So if you change one old block, its hash changes, then the next block's previous-hash reference breaks, then the next one, and so on. The lectures describe this as "unchaining."

Easy example

Suppose block 55 has:

- transaction: "Asim pays Arif \$20"

If someone changes it to:

- "Asim pays Arif \$50"

then block 55's hash changes. Since block 56 stores the old hash of block 55, the chain no longer matches. That is why tampering is easy to detect.

B. Nonce and mining

A block has transactions, previous block hash, and a **nonce**. Miners keep changing the nonce until the block hash satisfies the required condition. The lectures literally define mining as cycling through the nonce to meet some hash criteria, and the quiz also says the nonce's purpose is to provide a changing input during hashing.

Your exam language

Sir may phrase it as:

- valid if $H(\text{prev hash}, \text{Tx}, \text{nonce}) < \text{target}$
- or valid if block header hash is less than a target

The slides show exactly that kind of condition.

Important intuition

The nonce is not there for “meaning.”

It is there so miners can keep trying different hashes.

If sir asks:

Why do miners change nonce?

Because transaction data and previous hash are mostly fixed, so changing the nonce gives new hash attempts until the hash becomes smaller than the target.

C. Proof-of-work and target

Mining is proof-of-work. A miner searches over the nonce space until they find an eligible hash. It is hard to find but easy for others to verify.

The block header lecture gives the formal idea:

- block header has version, previous block hash, Merkle root, timestamp, target hash, nonce
- block header hash is $\text{SHA256}(\text{SHA256}(\text{Block_Header}))$

What target means

Target controls difficulty.

- **Smaller target** → harder
- **Larger target** → easier

Because the hash must be below the target.

Example

If target = 5000 and hashes are:

- 8123
- 7450
- 4998
- 5301

Only **4998** is valid because it is the only one below target.

4) How a block gets added

The lectures describe a full flow:

- user creates and broadcasts transaction
- validators/auditors keep valid unconfirmed transactions in their log/mempool
- miners choose transactions and form a candidate block
- miner solves proof-of-work
- broadcasts block
- others verify and accept it into their journal
- the converged result becomes the ledger

They also emphasize:

- a transaction becomes more final after more confirmations
- forks can happen temporarily
- longest chain gets picked

Quiz connection

One quiz asks what you should do if you receive another valid block while building your own.
Correct answer: **discard yours and start on top of the received valid block.**

That is very testable.

5) Bitcoin ownership in simple words

The lecture says Bitcoin is always **locked to the owner**. The payer unlocks it, then locks it again for the payee. Only the correct owner can unlock.

That one sentence explains almost the entire UTXO logic.

6) Bitcoin transaction model: the most important topic

A. UTXO model

Bitcoin uses **Unspent Transaction Outputs**.

A transaction:

- consumes old UTXOs as inputs
- creates new UTXOs as outputs

The rules lecture says:

- each output must have a value and a `scriptPubKey`
- each input must reference one old output
- in legacy txns the input contains `scriptSig`
- in SegWit txns `scriptSig` is empty and witness contains the unlocking data
- total input value must be at least total output value

Easy example

Parent transaction gives Alice **1.5 BTC** in output index 0.

Alice wants to pay Bob **0.5 BTC**.

She can create a child transaction that:

- references that parent output as input
- creates:
 - one output to Bob for 0.5 BTC
 - one change output back to herself for the remaining amount minus fee

The lecture explicitly shows this “change output” idea.

B. Why change output exists

If Alice owns a UTXO worth 1.5 BTC and only sends Bob 0.5 BTC with no change output, then the missing 1 BTC becomes fee to miner. The lecture directly gives this example and warns that without change output Alice loses the remaining amount as fee.

So remember:

Bitcoin does **not** partially spend one output.
It spends the **whole referenced output**, then creates fresh outputs.

This is one of the biggest conceptual points.

C. Transaction fee

The UTXO rules imply:

Fee = total inputs – total outputs

The lecture example shows:

- total input is computed from referenced parent outputs
- outputs are smaller
- difference is fee

Example

If inputs total = 0.01122918 BTC
and outputs total = 0.01119302 BTC
then fee = 0.00003616 BTC

That exact structure appears in the lecture example.

D. Fee rate

Your sir mentioned fee and fee rate.

Formula

Fee rate = fee / transaction size

Usually this is expressed in satoshis per byte or satoshis per vbyte.

Example

If fee = 200 sats and transaction size = 100 bytes
fee rate = 2 sats/byte

Why fee rate matters

Because miners have limited block space and prefer transactions that earn more per unit of space.

Even if the lecture doesn't spell out the formula in the snippet we saw, this follows directly from the fee concept and how block space is limited.

7) Bitcoin transaction format

The legacy Bitcoin transaction lecture shows the key fields:

Input

- previous transaction ID
- index
- scriptSig

Output

- value
- scriptPubKey

Other field

- lock_time

Meaning

- `Prev Txid` + `Index` tells **which old output** you are spending
 - `scriptSig` is the unlocking data
 - `scriptPubKey` on the output is the locking condition for whoever will spend it later
-

8) scriptSig and scriptPubKey

This is absolutely exam-worthy.

The lecture says:

- **scriptSig** encodes the payer's signature and public key
- **scriptPubKey** is a sequence of instructions that acts like a boolean predicate
- it takes the unlocking data and returns **True** if the spender is legitimate, else **False**

In plain words

- `scriptPubKey` = lock
- `scriptSig` = key + proof that you're allowed to open the lock

If the combined execution returns **True**, the output is unlocked and can be spent.

9) Signature construction and verification

This is another huge topic.

A. What is signed?

The transaction authorization part of the lecture shows that a transaction needs:

- payer
- payee
- amount
- authorization

And the authorization is conceptually a signature over the hashed transaction data.

Simple mental model

Alice signs a hash of the relevant transaction contents using her private key.

The network verifies using her public key.

B. What digital signature proves

The digital-signature lectures explain that hashing + signing gives:

- **authentication**: message came from the holder of the private key
- **integrity**: message was not changed afterward

Very important subtlety

Signing raw message directly is weaker; the lecture explains why hashing before signing avoids garbage-injection style problems.

C. Bitcoin-specific signature data in legacy transactions

The lecture says:

- `scriptSig` contains an **ECDSA signature** and a **public key**
- the signature is a pair `(r, s)`
- DER encoding format is shown
- sighash byte indicates which transaction parts were signed

You do **not** need to memorize every byte layout unless sir has practiced it in class, but do know:

Signature structure idea

- ECDSA signature = `(r, s)`
- stored in DER encoded form
- then a sighash flag is appended

Sighash byte

Slides mention:

- `0x01` = all inputs and all outputs hashed and signed

- `0x02` = all inputs hashed and signed but no output

That is very much the type of detail sir may use in a tricky conceptual question.

10) P2PKH: the standard legacy unlock flow

This is probably the most likely simulation question.

A. What P2PKH means

Pay to Public Key Hash

Instead of locking an output directly to a big public key, Bitcoin locks it to the **hash of the public key**, which is shorter. The lecture says pubkeys are large but pubkey hash is only 20 bytes.

B. Standard P2PKH locking script

The slides show:

```
OP_DUP OP_HASH160 <H(pubkey)> OP_EQUALVERIFY OP_CHECKSIG
```

C. Unlocking data

```
<signature> <public key>
```

D. Step-by-step execution in simple words

Imagine the stack initially gets:

- signature
- public key

Then:

1. `OP_DUP`

Duplicate the public key

Quiz 4 explicitly tested this: after `OP_DUP`, there are two identical public keys on the stack.

2. `OP_HASH160`

Hash the top public key

3. Push stored pubkey hash

This is the expected pubkey hash from the old output

4. `OP_EQUALVERIFY`

Check that:

- hash(provided pubkey)
- stored pubkey hash

are equal.

If not equal, fail.

5. `OP_CHECKSIG`

Use the remaining signature + public key to verify that the transaction was signed by the holder of the matching private key.

If valid, push True.

That is the P2PKH unlock idea.

What is being checked overall?

The lectures say the purpose is to check that the locked bitcoin in the previous transaction belongs to the current spender, and that `scriptSig + scriptPubKey` returns True.

11) SegWit and P2WPKH

SegWit was introduced to prevent transaction malleability.

A. Transaction malleability

The lecture explains:

- txid in legacy Bitcoin included `scriptSig`
- ECDSA signatures can have multiple valid encodings
- so the txid could change even though the transaction effect stayed the same

Why it is bad

- unreliable transaction IDs
- wallet confusion
- rebroadcasted modified versions
- layer-2 protocols break because they depend on stable txids

B. SegWit solution

The lecture says:

- `scriptSig = <signature, public key>` is really witness data
- separate this witness from the transaction input
- txid is then not created using scriptSig

C. P2WPKH format

In SegWit P2WPKH:

- input `scriptSig` is empty
- unlocking data is in the **witness**
- witness contains signature and public key

D. How P2WPKH unlocking works

The lecture says Bitcoin internally expands the witness program into a **virtual P2PKH script**:

`OP_DUP OP_HASH160 <pubKeyHash> OP_EQUALVERIFY OP_CHECKSIG`

So conceptually:

- same logical ownership test as P2PKH
- but witness is separated from txid computation

Easy exam line

P2WPKH is basically SegWit version of P2PKH.

12) Multisig, P2MS, P2SH, P2WSH

A. What multisig means

Multisig requires **M-of-N** signatures.

Example:

- 2-of-3 means any 2 valid signatures out of 3 listed public keys are enough.

Why useful

Lecture examples:

- shared accounts
 - company treasury
 - escrow
 - Lightning channels
-

B. P2MS

Pay to Multisig

Coins are locked to a script requiring multiple signatures.

Example locking script

```
OP_2 <PubKey1> <PubKey2> <PubKey3> OP_3 OP_CHECKMULTISIG
```

Meaning:

- there are 3 listed keys
 - require 2 valid signatures to spend
-

C. P2SH

Pay to Script Hash

Instead of putting a full complex script directly in the visible output, Bitcoin can lock to the **hash of a script**.

To spend it, the spender must provide:

- the redeem script
- the data needed to satisfy that script

This is why lecture slides say the multisig script is often hashed into a P2SH or P2WSH address.

Easy way to remember

P2SH = pay to the hash of some script
Later, reveal the script and satisfy it.

D. P2WSH

Pay to Witness Script Hash

This is the SegWit version of script-hash spending.

Lecture explains the P2WSH multisig execution:

1. Last witness item must be the witness script
2. Compute SHA256 of witness script
3. Compare it to the 32-byte value stored in previous output's scriptPubKey
4. If equal, continue
5. Push other witness items
6. Execute witness script
7. Verify M valid signatures against N public keys

Important detail

Slides even note the `OP_0` / CHECKMULTISIG bug behavior in multisig witness execution.

E. P2TR

Pay to Taproot

Your available lectures list P2TR as one of the possible Bitcoin script lock types.

For your midterm level, what you should know is:

- it is a newer Bitcoin transaction type
- it belongs to newer SegWit generation
- it supports more flexible spending conditions
- it improves efficiency and privacy compared to older forms

If sir asks transaction-type comparison, mention:

- legacy: P2PKH, P2MS, P2SH
 - SegWit v0: P2WPKH, P2WSH
 - SegWit v1 / Taproot: P2TR
-

13) Merkle tree and Merkle proof

This is another very likely long question.

A. Why Merkle tree is needed

If a block has many transactions, verifying one transaction by checking all transactions is expensive. The lecture shows this ordinary verification as $O(n)$ and Merkle proof as $O(\log n)$.

B. What Merkle tree is

- hash each transaction
- pair hashes and hash again
- continue upward
- topmost hash is the **Merkle root**
- the Merkle root is stored in the block header

C. Why it is useful

Lecture says Merkle tree:

- summarizes transactions
- enables lightweight verification / SPV
- detects tampering
- improves scalability because proof size is small

D. Merkle proof

To prove a transaction is inside a block, you do **not** need all transactions.
You only need:

- the hash of that transaction
- sibling hashes on the path to the root

The lecture explicitly says Merkle proof needs only about $\log_2(N)$ hashes.

Example with 4 transactions

Let:

- $h1 = H(Tx1)$
- $h2 = H(Tx2)$
- $h3 = H(Tx3)$
- $h4 = H(Tx4)$

Then:

- $h12 = H(h1 || h2)$
- $h34 = H(h3 || h4)$
- $root = H(h12 || h34)$

To prove Tx2 is included:

- start with h2
- need sibling h1
- then need sibling h34
- recompute root

If recomputed root matches block's Merkle root, inclusion is proven.

14) Block header: exact fields

The block header lecture lists 6 fields:

1. Version
2. Previous block hash
3. Merkle root hash
4. Timestamp
5. Target hash / difficulty target
6. Nonce

This is very important

If sir gives you a simulation around PoW or asks what changes during mining, remember:

- previous block hash links chain
 - Merkle root summarizes transactions
 - target defines difficulty
 - nonce is varied during search
-

15) UTXO database

This one is directly named in your exam topics.

The lecture says:

- nodes maintain a UTXO database called `chainstate.db`
- it speeds up transaction validation
- a UTXO is an output not yet spent

Simplified validation algorithm

For each input in a new transaction:

1. If referenced parent UTXO is not in database → error
2. If signature does not match owner of parent UTXO → error
3. If total parent UTXO value < total child outputs → error
4. If successful:
 - remove parent UTXOs
 - add child outputs as new UTXOs

This is one of the best exam answers you can memorize almost word for word.

16) Double spending and races

The lecture shows that two child transactions can race to spend the same output. The one that gets confirmed wins.

Important idea

Two different children spending **different indexes** of same parent tx are both valid.
But two children spending the **same exact UTXO** are in conflict.

17) Bitcoin scripts: what they can do

The lecture says Bitcoin opcodes can:

- check signatures
- combine conditions
- require multiple approvals
- lock funds by time
- lock funds by delay

This is useful if sir asks a short conceptual question like:

What can Bitcoin Script do besides simple payment?

18) Transaction types summary

This is the cleanest table for memorizing.

P2PKH

- Full form: Pay to Public Key Hash
- Locks to hash of public key
- Spend by giving signature + public key
- Standard legacy transaction

P2MS

- Full form: Pay to Multisig
- Locks to M-of-N multisig script
- Spend by giving required number of signatures

P2SH

- Full form: Pay to Script Hash
- Locks to hash of redeem script
- Spend by revealing redeem script and satisfying it

P2WPKH

- Full form: Pay to Witness Public Key Hash
- SegWit version of P2PKH
- Witness contains signature + public key
- scriptSig empty

P2WSH

- Full form: Pay to Witness Script Hash
- SegWit version of script-hash
- Witness contains witness script + witness data
- often used for SegWit multisig

P2TR

- Full form: Pay to Taproot
- newer SegWit v1/Taproot style
- more advanced, flexible, efficient spend conditions

19) Quiz-based facts you should memorize

The quizzes are useful because they show the exact style of distinctions sir likes.

From Quiz 1

- nonce provides changing input during hashing
- blockchain immutability is strongly tied to sensitivity of hash output to input modification
- halving = block reward reduced by 50% roughly every 210,000 blocks

From Quiz 2

- ledger is logically updated when the network reaches agreement on the proposed block, not just when transaction is first broadcast
- when you receive another valid block while constructing yours, build on top of it

From Quiz 4

- in P2PKH, scriptSig provides digital signature + corresponding public key
- after `OP_DUP`, the public key appears twice on stack
- if full UTXO value minus fees is being sent and no change remains, only **one output** is needed

These tiny points are exactly the kind of MCQs or short conceptual lines that can save marks.

20) Things students usually confuse

Confusion 1: Is Bitcoin account-based?

No. Bitcoin uses UTXOs. Ethereum uses account-based model.

Confusion 2: Can one output be partly spent?

Not directly. You spend the whole referenced output and then create new outputs, including change if needed.

Confusion 3: Is scriptSig present in SegWit?

Usually no. In SegWit, scriptSig is empty and unlocking data moves to witness.

Confusion 4: What exactly is checked in unlocking?

That the spender:

- referenced an existing UTXO
- provided correct unlocking data
- satisfies the locking script
- and input value covers outputs

Confusion 5: Does Merkle proof prove block validity too?

Not by itself. It proves transaction inclusion relative to a Merkle root. You still also care whether the block itself is valid and part of accepted chain. Lecture's verification steps distinguish those things.

21) Super useful answer templates

These are exam-style mini answers.

What is a UTXO?

A UTXO is a transaction output that has been created in a previous transaction and not yet spent. Bitcoin transactions consume old UTXOs as inputs and create new UTXOs as outputs.

What is scriptPubKey?

It is the locking script in an output that defines the condition required to spend that output later.

What is scriptSig?

It is the unlocking data in a legacy Bitcoin input, typically containing signature and public key, used to satisfy the previous output's locking script.

Why is SegWit used?

To separate witness data from transaction ID calculation and reduce transaction malleability.

What is Merkle root?

It is the top hash obtained by repeatedly hashing pairs of transaction hashes in a Merkle tree, and it summarizes all transactions in the block.

What is mining?

Mining is the process of varying nonce and hashing the block header until the resulting hash meets the difficulty target.

22) What to practice tonight

Do these in writing, not just reading:

Practice set 1: UTXO spending

Take one parent transaction and make:

- one payment with no change
- one payment with change
- one invalid payment where outputs exceed inputs

Practice set 2: P2PKH execution

Write the stack steps:

- signature, pubkey
- OP_DUP
- OP_HASH160
- compare hash
- OP_CHECKSIG

Practice set 3: Mining

Given 4 hash values and a target, choose valid nonce.

Practice set 4: Merkle tree

Make a 4-leaf Merkle tree and prove inclusion of Tx2.

Practice set 5: Transaction types

Write one line each for:

- P2PKH
- P2MS

- P2SH
 - P2WPKH
 - P2WSH
 - P2TR
-

23) Final ultra-short cram sheet

If you revise only one page before exam, memorize this:

- Bitcoin uses **UTXO**, not account balances.
- Input references old output using **Prev Txid + Index**.
- Output contains **Value + scriptPubKey**.
- Legacy input unlocks using **scriptSig**; SegWit uses **witness**.
- Transaction valid if:
 - referenced UTXO exists
 - unlock data matches owner
 - total inputs $\geq$ total outputs
- **Fee = inputs - outputs**
- **Fee rate = fee / size**
- P2PKH lock:


```
OP_DUP OP_HASH160 <pubKeyHash> OP_EQUALVERIFY OP_CHECKSIG
```
- P2PKH unlock:


```
<signature> <public key>
```
- **OP_DUP** duplicates the public key.
- SegWit prevents txid malleability by removing witness from txid.
- Merkle root summarizes all transactions.
- Merkle proof needs only about **$\log_2(N)$** hashes.
- Block header includes:

version, prev hash, Merkle root, timestamp, target, nonce.
- Mining = keep changing nonce until block hash < target.
- P2MS = M-of-N signatures.
- P2SH = pay to script hash.
- P2WPKH = SegWit P2PKH.
- P2WSH = SegWit script-hash.
- P2TR = Taproot.